

第 2 周：基础 API 与 MySQL

本周目标

完成任务管理的基础 API、MySQL 表结构和数据库访问层。

本周详细需求

数据库设计

sync_jobs（任务表）

字段	类型	说明
id	varchar(36)	主键，UUID 或 ULID
name	varchar(128)	任务名称，用户自定义或系统生成
status	varchar(32)	当前状态，本周先用 PENDING / RUNNING / SUCCESS / FAILED
source_file_name	varchar(255)	原始文件名
stored_file_path	varchar(512)	受控存储路径， 不通过 API 返回
file_sha256	char(64)	文件 SHA-256 摘要
idempotency_key	varchar(128)	幂等键，可为空（第 5 周启用）
total_records	int unsigned	有效数据行总数，默认 0
success_records	int unsigned	写入成功数，默认 0
failed_records	int unsigned	行级失败数，默认 0
retry_count	tinyint unsigned	已重试次数，默认 0（第 5 周启用）
last_error_code	varchar(64)	最近错误码，可为空
last_error_message	varchar(512)	最近错误摘要，可为空
created_at	datetime(3)	创建时间，UTC
started_at	datetime(3)	首次开始时间，UTC，可为空
finished_at	datetime(3)	完成时间，UTC，可为空

字段	类型	说明
updated_at	datetime(3)	更新时间，UTC

索引：主键 `id`；索引 `(status, created_at)`、`(created_at)`。

sync_records（成功记录表）

字段	类型	说明
id	bigint unsigned	自增主键
job_id	varchar(36)	所属任务
external_id	varchar(64)	外部业务标识，统一大写存储
name	varchar(128)	名称
amount	decimal(12,2)	金额，禁止使用 FLOAT/DOUBLE
record_date	date	业务日期
created_at	datetime(3)	创建时间
updated_at	datetime(3)	更新时间

约束： `UNIQUE(job_id, external_id)`；索引 `(job_id)`。

sync_errors（错误记录表）

字段	类型	说明
id	bigint unsigned	自增主键
job_id	varchar(36)	所属任务
row_number	int unsigned	CSV 行号；文件级错误可为空
field_name	varchar(64)	错误字段，可为空
error_code	varchar(64)	稳定错误码
error_message	varchar(512)	可读错误消息
raw_row	json	原始行摘要，可为空
created_at	datetime(3)	创建时间

索引： `(job_id, row_number)`。

迁移脚本能重复执行（跑两遍结果一样），集成测试用独立的测试数据库，别污染开发库。

API 设计

统一响应格式

成功响应：

```
{ "data": {}, "meta": {} }
```

失败响应：

```
{ "error": { "code": "ERROR_CODE", "message": "可读消息", "details": [] } }
```

本周错误码：

HTTP 状态	错误码	场景
400	INVALID_REQUEST	请求字段或分页参数非法
400	INVALID_FILE_EXTENSION	非 CSV 文件
400	FILE_TOO_LARGE	文件超过大小限制
404	JOB_NOT_FOUND	任务不存在
500	INTERNAL_ERROR	未预期内部错误

创建任务

```
POST /api/v1/jobs
Content-Type: multipart/form-data
```

字段	必填	说明
file	是	CSV 文件
name	否	用户自定义任务名称，最长 128 字符；不传时由系统生成

规则： – 文件扩展名为 `.csv`，否则返回 400 `INVALID_FILE_EXTENSION`。 – 文件大小默认上限 10 MB，配置项 `MAX_UPLOAD_FILE_SIZE_MB`，超过返回 400 `FILE_TOO_LARGE`。 – 文件存到本地受控目录，别直接拿客户端文件名当本地路径——有人会传 `../../xxx` 这种文件名做目录穿越。 – 算一下文件 SHA-256 存起来，后面幂等判断要用。 – 创建后状态是 `PENDING`，接口立即返回，别等文件处理完。 – 队列投递失败了要处理，任务不能一直卡在 `PENDING` 又没有任何错误记录。

成功返回 201 Created :

```
{
  "data": {
    "id": "job_01JABC...",
    "name": "商品导入-2026-01-01",
    "status": "PENDING",
    "created_at": "2026-01-01T10:00:00Z"
  }
}
```

查询任务详情

```
GET /api/v1/jobs/{job_id}
```

成功响应里要带上这些字段: `id`、`name`、`status`、`source_file_name`、`total_records`、`success_records`、`failed_records`、`retry_count`、`last_error_code`、`last_error_message`、`created_at`、`started_at`、`finished_at`。

- `job_id` 不存在返回 404 `JOB_NOT_FOUND`。
- 别返回 `stored_file_path`、数据库连接信息或内部堆栈, 这些是内部细节。

查询任务列表

```
GET /api/v1/jobs?page=1&page_size=20&status=SUCCESS
```

规则: - `page` 默认 1, 最小值 1。 - `page_size` 默认 20, 最大值 100。 - 支持按 `status` 筛选; 状态值非法返回 400 `INVALID_REQUEST`。 - 默认按 `created_at` DESC, `id` DESC 排序, 保证分页稳定。 - 返回列表、分页信息 (`page`、`page_size`、`total`) 和总数。

最小 Worker

本周需要一个能跑起来的最小 Worker, 不然创建的任务会一直卡在 `PENDING`。不要求多完善, 能走通就行:

- 从 Redis 队列里拿任务 ID。
- 拿到后把状态从 `PENDING` 改成 `RUNNING`, 记下 `started_at`。
- 本周 Worker 可以只做一件事: 把任务状态直接改成 `SUCCESS`, 模拟处理完成 (真正的 CSV 解析和写入在第 3 周实现)。
- 记下 `finished_at`。
- 日志带上 `job_id`。
- Worker 作为独立服务启动, 和 API 分开, 在 Docker Compose 里单独配一个服务。

这样 Week 2 结束时，创建任务 → 投队列 → Worker 消费 → 状态变 SUCCESS 的完整链路能跑通。第 3 周再把 Worker 里的"模拟完成"替换成真正的 CSV 解析和 MySQL 写入。

React 页面规格

任务列表页（ / ）

1. 首次加载调用 `GET /api/v1/jobs` 。
2. 支持按任务状态筛选，筛选条件与 URL 查询参数同步。
3. 支持分页，切换页码时保留筛选状态。
4. 每个任务展示：名称、状态、文件名、总数、成功数、失败数、创建时间、详情入口。
5. 状态用统一的中文展示名和颜色标识，但别只靠颜色区分——色弱的同学可能分不清，状态文字要始终在。
6. 没任务时显示空状态和"创建任务"按钮；网络挂了显示重试按钮，别白屏。

任务详情页（ /jobs/:jobId ）

1. 首次加载调用 `GET /api/v1/jobs/{job_id}` 。
2. 展示：任务状态、源文件名、总记录数、成功数、失败数、创建/开始/结束时间、最近错误。
3. `job_id` 不存在时显示"任务不存在"和返回列表的入口，别白屏也别崩。
4. 时间按浏览器本地时区显示，格式统一成 `2026-01-01 18:00:00` 这样。

通用要求

- API 响应类型集中定义，别每个页面自己写一套。
- 每个页面都要处理加载中、空数据、请求失败三种状态。
- 未匹配的路由（*）显示 404 页面，提供返回任务列表的入口，别白屏。
- 生产构建要能过，TypeScript 类型错误清零。

任务

1. 编写数据库迁移或初始化脚本。
2. 创建 `sync_jobs`、`sync_records`、`sync_errors` 表。
3. 实现 MySQL 连接、配置和健康检查。
4. 实现创建任务接口。
5. 实现任务详情接口。
6. 实现任务列表接口。
7. 实现最小 Worker（从队列取任务、更新状态为 SUCCESS）。

- 8. 增加请求参数校验。
- 9. 设计统一错误响应。
- 10. 实现 React 任务列表页和基础任务详情页。
- 11. 为正常和异常场景编写测试。

产出物

- 任务管理后端：MySQL 迁移、数据访问层、创建/详情/列表 API 和 OpenAPI 文档。
- 任务查询 Web 功能：React 任务列表、基础详情页、前端 API 客户端和页面状态处理。
- 可验证交付记录：单元测试、MySQL 集成测试、至少 2 个 PR 及其 Review。

产出物细则

产出物	必须包含	完成判定
任务管理后端	可重复迁移、数据访问层、创建/详情/列表接口、分页、统一错误和 OpenAPI	空数据库可初始化；接口符合状态码约定；路由层不直接承担 SQL 或业务规则
任务查询 Web 功能	列表、状态筛选、分页、详情、加载/空/错误/404 状态和 API 类型定义	浏览器调用真实 API，筛选条件与 URL 同步，接口失败不导致页面崩溃
可验证交付记录	成功、参数异常、数据库异常、分页边界的测试；PR 与 Review	测试不污染开发数据库；提交可独立审阅与合并

验收标准

- 数据库可以重复初始化。
- API 不直接暴露数据库异常堆栈。
- 查询结果字段命名统一。
- 无效参数返回明确的错误码和消息。
- 测试可以使用独立测试数据库。
- 能够解释路由层、业务层和数据访问层的职责。